

LAB REPORT 03

The Selection Structure in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The aim of this lab was to understand how C++ programs make decisions at runtime using the **selection structure**. The session introduced comparative (relational) operators and three conditional constructs — **if**, **if-else**, and **nested if-else** — working toward the following outcomes:

- Understanding relational operators and the boolean values they produce
- Writing a basic **if** statement that executes a block only when a condition is true
- Using **if-else** to choose between exactly two alternate execution paths
- Applying **nested if-else** ladders to handle more than two possible outcomes
- Recognising common mistakes such as `=<` instead of `<=`, or `=` instead of `==`

2 Theoretical Background

Comparative (Relational) Operators

Relational operators compare two values and evaluate to either **true** or **false**. They form the conditions driving every selection structure. The six operators available in C++ are listed below.

C++ Symbol	Math Notation	Meaning
<code><</code>	$<$	Less than
<code>></code>	$>$	Greater than
<code><=</code>	$\leq$	Less than or equal to
<code>>=</code>	$\geq$	Greater than or equal to
<code>==</code>	$=$	Equal to
<code>!=</code>	$\neq$	Not equal to

Common operator pitfalls:

- Write `<=` not `=<` — the less-than symbol must come first
- Use `==` to test equality; a single `=` performs assignment, not comparison
- Floating-point values may not satisfy `==` due to rounding errors
- A string literal cannot be compared to a number with a relational operator

The if Statement

The **if** statement evaluates a condition. When **true**, the statements inside the braces execute; when **false**, the entire body is skipped and execution resumes at the next statement after the block. Curly braces are optional for a single-statement body, though omitting them is generally discouraged as it invites errors.

The if-else Statement

An **else** clause provides a second path. Exactly one branch always executes: the **if**-block when the condition is **true**, the **else**-block when it is **false**. This guarantees an action is taken regardless of the input.

Nested if-else (else-if Ladder)

When more than two outcomes exist, **else if** chains additional conditions. The runtime evaluates conditions in order and executes only the first matching branch. A trailing **else** acts as a catch-all. The same result can also be achieved with independent **if** statements using the **&&** operator to define explicit ranges.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Basic if Statement: Age Check

A program reads the user's age and prints a message only if the age exceeds 18. When the condition is false the body is skipped entirely, and a final statement always prints to confirm execution continues normally after the **if** block.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main()
{
    int age;
    cout << "Enter your age in years: ";
    cin >> age;

    if (age > 18)
    {
        cout << "You are adult";
    }

    cout << "This program demonstrates the if statement";
    return 0;
```

}

Task 2 — if-else Statement: Adult or Minor

The program was extended with an **else** clause so that a different message prints when age is 18 or below. This ensures meaningful output regardless of input.

● ● ● C++ Code

```

#include <iostream>
using namespace std;
int main()
{
    int age;
    cout << "Enter your age in years: ";
    cin >> age;

    if (age > 18)
    {
        cout << "You are adult" << endl;
        cout << "You can cast vote in the election";
    }
    else
    {
        cout << "You are not adult" << endl;
        cout << "You are not allowed at the polling station";
    }

    cout << "This program demonstrates the if-else statement";
    return 0;
}

```

Task 3 — Nested if-else: Number Comparison

A nested **else-if** ladder determines whether a number entered by the user is greater than, equal to, or less than 10. Three mutually exclusive branches cover all cases.

● ● ● C++ Code

```

#include <iostream>
using namespace std;
int main()
{
    int i;
    cout << "Type any number: ";
    cin >> i;

    if (i > 10)
    {

```

```

        cout << "It's greater than 10." << endl;
    }
    else if (i == 10)
    {
        cout << "It's equal to 10" << endl;
    }
    else
    {
        cout << "It's less than 10." << endl;
    }
    return 0;
}

```

Task 4 — Grade Assignment — Descending else-if Ladder

An else-if ladder assigns a letter grade to a numeric score, checking from highest to lowest. Once a matching branch executes, the rest are bypassed.

● ● ● C++ Code

```

#include <iostream>
using namespace std;
int main()
{
    int score;
    cout << "Enter your score (0 to 100): ";
    cin >> score;

    if (score >= 90)      cout << "A";
    else if (score >= 80) cout << "B";
    else if (score >= 70) cout << "C";
    else if (score >= 60) cout << "D";
    else                  cout << "F";

    return 0;
}

```

Task 5 — Grade Assignment — Ascending else-if Ladder

The same grading logic was rewritten checking from the lowest score upward. Both approaches produce identical output, demonstrating that the direction of checking can be reversed as long as threshold values are adjusted accordingly.

● ● ● C++ Code

```

#include <iostream>
using namespace std;
int main()
{

```

```

int score;
cout << "Enter your score (0 to 100): ";
cin >> score;

if (score < 60)      cout << "F";
else if (score < 70) cout << "D";
else if (score < 80) cout << "C";
else if (score < 90) cout << "B";
else                cout << "A";

return 0;
}

```

Task 6 — Grade Assignment — Independent if Statements with &&

An alternative approach uses independent **if** statements with the logical AND operator to specify explicit score ranges. Every condition is evaluated each time, but because the ranges are mutually exclusive only one branch ever executes.

● ● ● C++ Code

```

#include <iostream>
using namespace std;
int main()
{
    int score;
    cout << "Enter your score (0 to 100): ";
    cin >> score;

    // Without nested structure
    if (score >= 90 && score <= 100) cout << "A";
    if (score >= 80 && score < 90)   cout << "B";
    if (score >= 70 && score < 80)   cout << "C";
    if (score >= 60 && score < 70)   cout << "D";
    if (score < 60)                 cout << "F";

    return 0;
}

```

5 Conclusion

This lab provided hands-on experience with the three core selection structures in C++. Starting with the relational operators that form the backbone of every condition, the tasks progressed from a simple **if** check through a two-way **if-else** decision to multi-branch **nested if-else** ladders.

The grade-assignment tasks highlighted an important design choice: an **else-if** ladder short-circuits as soon as one condition matches, while a sequence of independent **if** statements evaluates every condition every time. For mutually exclusive ranges both yield the same result, but the ladder communicates intent more clearly and

executes more efficiently. The exercises also reinforced two common pitfalls — writing `=<` instead of `<=`, and using `=` where `==` is required — mistakes that the compiler either flags as errors or, worse, silently misinterprets.

Mastering the selection structure is a fundamental step toward writing programs that respond intelligently to different inputs. The patterns practised here — particularly the else-if ladder — recur constantly in real-world C++ development.